

深圳大学实验报告

课程名称: 计算机系统(2)

实验项目名称: Cache 实验

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 马晨琳

报告人: 黄嘉聪 学号: 2023192044 班级: 1

实验时间: 2025 年 5 月 14 日至 5 月 27 日

实验报告提交时间: 2025 年 5 月 27 日

教务处制

一、实验目的：

1. 加强对 Cache 工作原理的理解；
2. 体验程序中访存模式变化是如何影响 cache 效率进而影响程序性能的过程；
3. 学习在 X86 真实机器上通过调整程序访存模式来探测多级 cache 结构以及 TLB 的大小。

二、实验环境

X86 真实机器

三、实验内容和步骤

1、分析 Cache 访存模式对系统性能的影响

- (1) 给出一个矩阵乘法的普通代码 A，设法优化该代码，从而提高性能。
- (2) 改变矩阵大小，记录相关数据，并分析原因。

2、编写代码来测量 x86 机器上（非虚拟机）的 Cache 层次结构和容量

- (1) 设计一个方案，用于测量 x86 机器上的 Cache 层次结构，并设计出相应的代码；
- (2) 运行你的代码获得相应的测试数据；
- (3) 根据测试数据来详细分析你所用的 x86 机器有几级 Cache，各自容量是多大？
- (4) 根据测试数据来详细分析 L1 Cache 行有多少？

4、尝试测量你的 x86 机器 TLB 有多大？

代码 A：

```
#include <sys/time.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    float *a,*b,*c, temp;
    long int i, j, k, size, m;
    struct timeval time1,time2;

    if(argc<2) {
        printf("\n\tUsage:%s <Row of square matrix>\n",argv[0]);
        exit(-1);
    } //if

    size = atoi(argv[1]);
    m = size*size;
    a = (float*)malloc(sizeof(float)*m);
    b = (float*)malloc(sizeof(float)*m);
```

```

c = (float*)malloc(sizeof(float)*m);

for(i=0;i<size;i++) {
    for(j=0;j<size;j++) {
        a[i*size+j] = (float)(rand()%1000/100.0);
        b[i*size+j] = (float)(rand()%1000/100.0);
    }
}

gettimeofday(&time1,NULL);
for(i=0;i<size;i++) {
    for(j=0;j<size;j++) {
        c[i*size+j] = 0;
        for (k=0;k<size;k++)
            c[i*size+j] += a[i*size+k]*b[k*size+j];
    }
}
gettimeofday(&time2,NULL);

time2.tv_sec-=time1.tv_sec;
time2.tv_usec-=time1.tv_usec;
if (time2.tv_usec<0L) {
    time2.tv_usec+=1000000L;
    time2.tv_sec-=1;
}

printf("Executiontime=%ld.%06ld seconds\n",time2.tv_sec,time2.tv_usec);
return(0);
} //main

```

四、实验结果及分析

1、分析 Cache 访存模式对系统性能的影响

①首先对原有的 a.cpp 进行优化。数组是以先行后列的形式逐个排列在内存中的，故我们在读取时，如果是逐行读取而不是逐列读取，则每次从读取的内容大概率会在上一次读取的内容的 Cache 缓存中，从而大幅度提高缓存命中率，可以很好的利用 Cache 缓存。

②但矩阵乘法本身是对应的行和列相乘，无法对两个矩阵都改成行相乘的模式，故我们先对其中一个矩阵进行转置，使得其原有需要读取的列变为行，然后再让两个矩阵使用行读取的形式进行矩阵乘法即可。

③除此之外，我们可以基于 Cache 的原理，将矩阵划分为多个 Cache 行大小的小块，从而使得代码的局部性更充分，提高程序的运行效率。优化完之后代码如图 1 所示。运行效率测试如表一所示

优化代码：

```

1  #include <sys/time.h>
2  #include <unistd.h>
3  #include <stdlib.h>
4  #include <stdio.h>
5  #include <algorithm>
6  #define BLOCK_SIZE 64
7
8  int main(int argc, char *argv[])
9  {
10     float *a, *b, *c, *b_transpose;
11     long int i, j, k, size, m;
12     struct timeval time1, time2;
13
14     if(argc<2) {
15         printf("\n\tUsage:%s <Row of square matrix>\n", argv[0]);
16         exit(-1);
17     }
18
19     size = atoi(argv[1]);
20     m = size*size;
21     a = (float*)malloc(sizeof(float)*m);
22     b = (float*)malloc(sizeof(float)*m);
23     c = (float*)malloc(sizeof(float)*m);
24     b_transpose = (float*)malloc(sizeof(float)*m);
25
26     // 初始化矩阵
27     for(i=0; i<size; i++) {
28         for(j=0; j<size; j++) {
29             a[i*size+j] = (float)(rand()%1000/100.0);
30             b[i*size+j] = (float)(rand()%1000/100.0);
31             c[i*size+j] = 0.0f;
32         }
33     }
34
35     // 转置矩阵b来提高缓存局部性
36     for(i=0; i<size; i++) {
37         for(j=0; j<size; j++) {
38             b_transpose[j*size+i] = b[i*size+j];
39         }
40     }
41
42     gettimeofday(&time1, NULL);
43
44     // 外层循环按块遍历
45     for (i = 0; i < size; i += BLOCK_SIZE) {
46         for (j = 0; j < size; j += BLOCK_SIZE) {
47             for (k = 0; k < size; k += BLOCK_SIZE) {
48                 // 计算当前块的实际大小（处理边界情况）
49                 int i_end = std::min(i + BLOCK_SIZE, size);
50                 int j_end = std::min(j + BLOCK_SIZE, size);
51                 int k_end = std::min(k + BLOCK_SIZE, size);
52
53                 // 计算子块的矩阵乘法
54                 for (int ii = i; ii < i_end; ii++) {
55                     for (int jj = j; jj < j_end; jj++) {
56                         float sum = c[ii*size+jj];
57                         for (int kk = k; kk < k_end; kk++) {
58                             // 利用转置矩阵提高内层循环的缓存命中率
59                             sum += a[ii*size+kk] * b_transpose[jj*size+kk];
60                         }
61                         c[ii*size+jj] = sum;
62                     }
63                 }
64             }
65         }
66     }
67
68     gettimeofday(&time2, NULL);
69
70     time2.tv_sec -= time1.tv_sec;
71     time2.tv_usec -= time1.tv_usec;
72     if (time2.tv_usec < 0) {
73         time2.tv_usec += 1000000L;
74         time2.tv_sec -= 1;
75     }
76
77     printf("Executiontime=%ld.%06ld seconds\n", time2.tv_sec, time2.tv_usec);
78
79     free(a);
80     free(b);
81     free(c);
82     free(b_transpose);
83
84     return 0;
85 }

```

图 1 优化过后的矩阵乘法

矩阵大小	100	500	1000	1500	2000	2500	3000
一般算法 执行时间	0.0021 40 秒	0.3139 13 秒	2.9358 89 秒	14.021 960 秒	26.693 109 秒	75.844 532 秒	125.75 506 秒
优化算法 执行时间	0.0017 30 秒	0.2632 78 秒	2.0392 37 秒	6.6833 19 秒	11.911 616 秒	33.672 514 秒	52.438 825 秒
加速比 speedup	1.2374	1.5324	1.4397	2.0981	2.2409	2.2524	2.3981

表 1、普通矩阵乘法与及优化后矩阵乘法之间的性能对比

加速比定义：加速比=优化前系统耗时/优化后系统耗时；

所谓加速比，就是优化前的耗时与优化后耗时的比值。加速比越高，表明优化效果越明显。

可以画出图像：

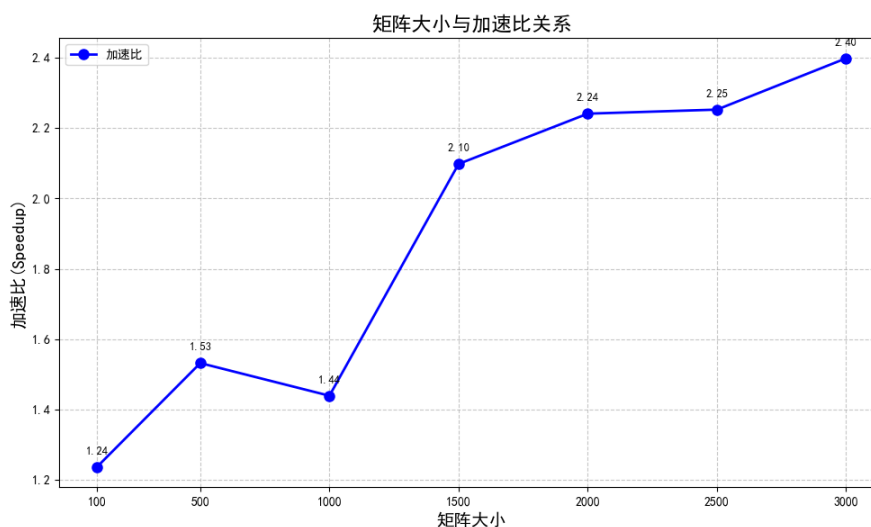


图 2 矩阵大小与加速比的关系

分析原因：①可以看到，在优化之后由于其局部性增强，其缓存命中率提高，使得代码运行明显加速。

②而对于更大的矩阵，由于多重缓存的存在，使得局部性增强本身不仅仅提高了其缓存命中率，同时也减少了其向高级缓存访问的次数，使得加速比随着矩阵大小逐渐上升。

2、测量分析出 Cache 的层次结构、容量以及 L1 Cache 行有多少？

（1）实验原理：对于当前的计算机 CPU，其 Cache 分为 3 层，分别为 L1、L2、L3，根据 Cache 的读取性质，层数越高的 Cache 的读取速度越慢，因此对于每一个层级的数据进行存取时，其速率之间会有较大的不同，我们可以从这种读取时间上的不同，获知其读取大小的临界点，从而大概得出各层级的 cache 大小。

（2）测量方案及代码：本次实验当中，通过两种策略分别测量 Cache 的层次结构、容量和 L1 Cache 行的大小。

①测量 Cache 的层次结构、容量：通过构造从 1KB 到 64MB 不同大小的工作集，并测量访问延迟。当工作集大小超过某一级缓存容量时，其访问延迟会陡然增加，通过观察其访问延迟即可获知其各级缓存容量

代码：

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <stdint.h>
5
6  double now_sec() {
7      struct timeval tv;
8      gettimeofday(&tv, NULL);
9      return tv.tv_sec + tv.tv_usec * 1e-6;
10 }
11
12 int main() {
13     const size_t max_size = 64 * 1024 * 1024; // 64 MB
14     const size_t line_size = 64;
15     const int repeats = 5;
16
17     printf("working_set(KB)\tavg latency (ns)\n");
18     for (size_t S = 1 << 10; S <= max_size; S <= 1) {
19         double total_ns = 0;
20         for (int r = 0; r < repeats; r++) {
21             uint8_t *A = (uint8_t *)malloc(S);
22             if (!A) { perror("malloc"); return 1; }
23             // 预热
24             for (size_t i = 0; i < S; i += line_size) A[i]++;
25
26             double t0 = now_sec();
27             size_t accesses = 0;
28             volatile uint8_t sink;
29             for (size_t i = 0; i < S; i += line_size) {
30                 sink = A[i];
31                 accesses++;
32             }
33             double t1 = now_sec();
34             total_ns += (t1 - t0) * 1e9 / accesses;
35             free(A);
36         }
37         printf("%12zu\t%12.2f\n", S / 1024, total_ns / repeats);
38     }
39     return 0;
40 }

```

图 3 Cache 测试代码

②L1 Cache 行的大小：通过按不同步长（4B 到 1024B）访问内存，当步长超过缓存行大小时，由于此时 Cache 需要使用下一级缓存，此时访问延迟会迅速上升，观察可获知其 L1 Cache 行的大小。

代码：

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <stdint.h>
5
6  #define ARRAY_SIZE (32 * 1024 * 1024)
7
8  double now_sec() {
9      struct timeval tv;
10     gettimeofday(&tv, NULL);
11     return tv.tv_sec + tv.tv_usec * 1e-6;
12 }
13
14 int main() {
15     uint8_t *array = (uint8_t *)malloc(ARRAY_SIZE);
16     if (!array) {
17         perror("malloc");
18         return 1;
19     }
20
21     printf("stride(bytes)\tavg latency (ns)\n");
22     for (size_t stride = 4; stride <= 1024; stride *= 2) {
23         // 预热
24         for (size_t i = 0; i < ARRAY_SIZE; i += stride) {
25             array[i]++;
26         }
27         // 测量
28         double t0 = now_sec();
29         size_t accesses = 0;
30         volatile uint8_t sink;
31         for (size_t i = 0; i < ARRAY_SIZE; i += stride) {
32             sink = array[i];
33             accesses++;
34         }
35         double t1 = now_sec();
36         double avg_ns = (t1 - t0) * 1e9 / accesses;
37         printf("%8zu\t%10.2f\n", stride, avg_ns);
38     }
39
40     free(array);
41     return 0;
42 }

```

图 4 Cache line 测试代码

(3) 测试结果:

①Cache 测试结果:

working_set(KB)	avg latency (ns)
1	3.58
2	1.19
4	1.94
8	3.58
16	2.79
32	2.72
64	1.30
128	1.12
256	1.11
512	1.05
1024	1.35
2048	5.86
4096	5.95
8192	6.36
16384	7.35
32768	9.13
65536	7.83

图 5 Cache 测试结果

②L1 Cache Line 长度测试结果:

stride(bytes)	avg latency (ns)
4	1.44
8	1.64
16	1.58
32	2.41
64	2.84
128	5.30
256	5.30
512	5.22
1024	5.58

图 6 Cache line 测试结果

(4) 分析过程: 由图 4 可知, 其访问时间在 1MB-2MB, 8MB-16MB, 16MB-32MB 处发生突变, 故可以推断出其有三层缓存, 并且大小分别为 1MB、8MB、16MB。

由图 5 可知, 其访问时间在 64B-128B 处发生突变, 故可以推断出其 L1 Cache 的长度为 64B。

(5) 验证实验结果: 如图 x 所示, 其为本次实验中使用的计算机的缓存大小, 和 (3) 中测试结果相同, 说明缓存测量成功。

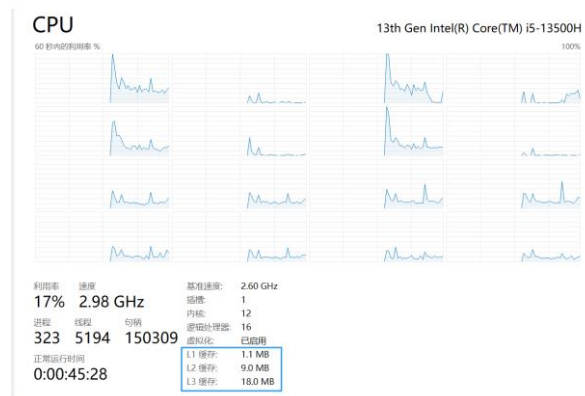


图 7 CPU 上的缓存验证测试结果

3、尝试测量你的 x86 机器 TLB 有多大？

(1)实验原理：TLB 是 CPU 中的一个特殊缓存，用于加速虚拟地址到物理地址的转换过程，可以使用类似于 Cache Line 的方法进行测量

(2)测量方法及原理：同 Cache Line 的方法，对不同大小的 TLB 进行访问测试并记录时间

代码：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4  #include <windows.h>
5
6  double now_sec() {
7      LARGE_INTEGER frequency, counter;
8      QueryPerformanceFrequency(&frequency);
9      QueryPerformanceCounter(&counter);
10     return (double)counter.QuadPart / frequency.QuadPart;
11 }
12
13 int main() {
14     // 获取系统页面大小
15     SYSTEM_INFO sysInfo;
16     GetSystemInfo(&sysInfo);
17     const size_t page_size = sysInfo.dwPageSize;
18
19     const size_t max_pages = 4096; // 最多测试到 4096 页
20     const int repeats = 5;
21
22     printf("num_pages\tlatency (ns)\n");
23     for (size_t np = 8; np <= max_pages; np *= 2) {
24         double total_ns = 0;
25         for (int r = 0; r < repeats; r++) {
26             size_t bytes = np * page_size;
27             uint8_t *A = (uint8_t*)VirtualAlloc(NULL, bytes,
28                                                 MEM_COMMIT | MEM_RESERVE,
29                                                 PAGE_READWRITE);
30
31             if (A == NULL) {
32                 fprintf(stderr, "VirtualAlloc failed: %lu\n", GetLastError());
33                 return 1;
34             }
35
36             // 预热
37             for (size_t i = 0; i < bytes; i += page_size)
38                 A[i]++;
39
40             double t0 = now_sec();
41             size_t accesses = 0;
42             volatile uint8_t sink;
43             for (size_t i = 0; i < bytes; i += page_size) {
44                 sink = A[i];
45                 accesses++;
46             }
47             double t1 = now_sec();
48             total_ns += (t1 - t0) * 1e9 / accesses;
49
50             VirtualFree(A, 0, MEM_RELEASE);
51         }
52         printf("%9zu\t%12.2f\n", np, total_ns / repeats);
53     }
54     return 0;
55 }
```

图 8 TLB 测试代码

(3) 测试结果与分析：

num_pages	latency (ns)
8	15.00
16	1.25
32	1.88
64	1.56
128	2.97
256	4.61
512	3.79
1024	4.98
2048	7.41
4096	13.76

图 9 TLB 测试结果

可以看的其在 64 页-128 页、1024 页-2048 页有明显增幅，故可以推断出其有两段 TLB，第一段的页数为 64，第二段的页数为 1024。

五、实验结论与心得体会

①通过本次实验，对计算机系统中 Cache 与 TLB 的层级结构及其所带来的性能优化进行了探索，结合本次实验的实验数据与相关的理论分析验证了多级缓存的工作机制及其对程序性能的直接影响。

②在实验当中，通过逐步增大工作集规模并测量访问延迟，成功捕捉到 L1、L2、L3 三级缓存的容量边界为 1MB、8MB、16MB。并通过计算机中所提供的缓存验证了结果。同时，针对 L1 Cache 行大小的探测实验显示，当访问步长超过 64B 后延迟陡增，这与 X86 架构中标准 Cache Line 尺寸一致，进一步证明了实验方法的有效性。

③在 TLB 特性分析中，通过分页访问模式的测试发现，当访问页数超过 64 页和 1024 页时地址转换延迟出现分段提升，成功测量出了实验机器的 TLB 为第一级 64 页，第二级 1024 页。

④此外，矩阵乘法优化实验直观展示了访存模式对性能的关键影响，通过循环重排与分块计算策略，使数据局部性得到显著改善，尤其在大规模矩阵运算中（如 3000×3000 规模）优化算法相较原始实现取得了 2.39 倍的加速比，说明了程序开放当中系统级视角的重要程度。

⑤最后，在本次实验当中，通过对比理论模型与实测数据的吻合性（如 CPU 缓存规格的交叉验证），强化了对存储层次结构特性的认知，还深化了对 Cache 和 TLB 工作机制的理解，更通过矩阵的性能改进认识到系统级视角在程序开发中的重要性，为后续代码的编写优化提供了重要的视角与方法。

指导教师批阅意见：

成绩评定：

指导教师签字：

2024 年 月 日

备注：